

105.2: Scripts shell

Rafael Obelheiro

OTCC Linux

105.2: Scripts shell

- Lição 1
 - execução de scripts (shebang, interpretador, permissões)
 - variáveis, variáveis internas do shell, read, vetores (Bash), comprimento de variáveis e vetores
 - expressões aritméticas
 - execução condicional (&&, ||, if/then/else)
 - echo, printf
- Lição 2
 - test, case
 - laços: for, while, until

Desenvolvimento de scripts

- Scripts raramente são desenvolvidos do zero diretamente como scripts
- Mais comum é agrupar sequências de comandos que realizam tarefas repetidas
 - muitas vezes acrescentando parametrização, tratamento de erros, formatação da saída, etc.

Desenvolvimento incremental de scripts

1. Desenvolver o script (ou partes dele) como um pipeline, um passo de cada vez, inteiramente na linha de comando
 - usando Bash para este processo
2. Enviar a saída para a saída padrão e verificar se está tudo certo
 - em vez de executar diretamente os comandos, usar echo para mostrar como eles ficarão
3. A cada passo, usar o histórico de comandos do shell para recuperar pipelines anteriores e os mecanismos de edição para refiná-los
4. Enquanto a saída não parecer correta, nada foi realmente executado, e não haverá nada a desfazer caso o comando esteja incorreto
5. Quando a saída estiver correta, executar os comandos e verificar que eles funcionam conforme o desejado
 - removendo os echos de depuração
6. Recuperar o que foi feito (usando o histórico), deixar organizado e guardar como um script

Tratamento de erros

```
#!/bin/sh

show_usage() {
    echo "Usage: $0 source_dir dest_dir" 1>&2
    exit 1
}

# Main program starts here

if [ $# -ne 2 ]; then
    show_usage
else # There are two arguments
    if [ -d $1 ]; then
        source_dir=$1
    else
        echo 'Invalid source directory' 1>&2
        show_usage
    fi
    if [ -d $2 ]; then
        dest_dir=$2
    else
        echo 'Invalid destination directory' 1>&2
        show_usage
    fi
fi

printf "Source directory is ${source_dir}\n"
printf "Destination directory is ${dest_dir}\n"
```

Argumentos de linha de comando

- O comando `shift [N]` remove o primeiro argumento (ou mais, se N for especificado)
 - o número de argumentos (`$#`) é decrementado de N
- É recomendável usar aspas ao referenciar os argumentos
 - minimiza problemas com argumentos contendo espaços ou não informados
- Exemplo: listar todos os argumentos passados para um script

```
i=0
while [ $# -gt 0 ]; do
    ((i++))
    printf "%2d: %s\n" $i "$1"
    shift
done
```

Shebang e variações de nome de caminho

- A especificação de interpretador de comandos para scripts (`#!`) requer um nome de caminho absoluto
- Especialmente no Linux, o Bash costuma residir em `/bin`
- Em alguns sistemas, porém, o Bash pode ser encontrado em outros diretórios, notadamente `/usr/local/bin`
- Para outros interpretadores de comandos (como Python ou Ruby), a variação é maior
- Uma forma de minimizar tais variações é usar `env`:

```
#!/usr/bin/env bash
```

 - isso invoca o primeiro `bash` no `PATH`
- Possíveis problemas
 - suscetível a incompatibilidade de versões de interpretadores
 - não é garantido que `env` resida em `/usr/bin`

if/then

- O comando `if` executa o comando passado como argumento
 - `if` trata como verdadeiro se o valor de retorno do comando for 0, e falso caso contrário
- Exemplo simples:

```
if ls /tmp >/dev/null 2>&1 ; then
    echo existe
else
    echo nao existe
fi
```
- O comando `test` é que testa condições lógicas e define um valor de retorno coerente com elas

Flags comuns de test (1)

Elementary sh comparison operators

String	Numeric	True if
<code>x = y</code>	<code>x -eq y</code>	x is equal to y
<code>x != y</code>	<code>x -ne y</code>	x is not equal to y
<code>x <^a y</code>	<code>x -lt y</code>	x is less than y
<code>n/a</code>	<code>x -le y</code>	x is less than or equal to y
<code>x >^a y</code>	<code>x -gt y</code>	x is greater than y
<code>n/a</code>	<code>x -ge y</code>	x is greater than or equal to y
<code>-n x</code>	<code>n/a</code>	x is not null
<code>-z x</code>	<code>n/a</code>	x is null

a. Must be backslash-escaped or double bracketed to prevent interpretation as an input or output redirection character.

Flags comuns de test (2)

sh file evaluation operators

Operator	True if
<code>-d file</code>	<i>file</i> exists and is a directory
<code>-e file</code>	<i>file</i> exists
<code>-f file</code>	<i>file</i> exists and is a regular file
<code>-r file</code>	User has read permission on <i>file</i>
<code>-s file</code>	<i>file</i> exists and is not empty
<code>-w file</code>	User has write permission on <i>file</i>
<code>file1 -nt file2</code>	<i>file1</i> is newer than <i>file2</i>
<code>file1 -ot file2</code>	<i>file1</i> is older than <i>file2</i>

if/elif/else

- Além de if/else, existe if/elif/else

- Exemplo simples:

```
if [ "$1" -lt 0 ] 2>/dev/null ; then
    echo "negativo"
elif [ "$1" -gt 0 ] 2>/dev/null ; then
    echo "positivo"
elif [ "$1" -eq 0 ] 2>/dev/null ; then
    echo "zero"
else
    echo "erro no argumento" >&2
    exit 1
fi
```

Obtendo entrada: read

- O comando interno read pode ser usado para ler uma entrada

```
echo -n "Digite seu nome: "
read nome
read -p "Digite sua idade: " idade
read -p "Digite sua cidade natal: "
echo "${nome} nasceu em ${REPLY}, e tem ${idade} anos.
```

- ▶ \$REPLY é específico do Bash

- Outras opções úteis

opção	significado
<code>-s</code>	não mostra caracteres
<code>-r</code>	preserva \ na entrada
<code>-n 1</code>	retorna após ler 1 caractere
<code>-t 5</code>	retorna erro se usuário não responder em 5 seg

Depurando scripts

- A opção `-n` faz com que o shell apenas analise a sintaxe do script, sem executar os comandos
- A opção `-x` faz com que o shell mostre os comandos que são executados
- O mesmo efeito pode ser obtido usando `set -x`
 - ▶ válido apenas no shell que executa o script
 - ▶ se necessário, `set +x` restabelece o comportamento padrão

Aritmética inteira no shell

- Forma mais portátil:

```
expr 3 + 4
i=$(expr $i + 1)
x=`expr 3 \* $y + $z / 4`
```

- Bash oferece uma notação mais simples:

```
$(( 3+4 )) (( i++ )) (( x=3*y + z/4 ))
```

case

- Os padrões usados no comando `case` podem usar metacaracteres
 - ▶ é usado *globbing* para o casamento de padrões
- Exemplo simples:

```
read -n1 -p "Pressione uma letra ou digito: " opcao
echo
case $opcao in
  [0-9])
    echo "Voce pressionou o digito $opcao" ;;
  [a-z]|[A-Z])
    echo "Voce pressionou a letra $opcao" ;;
  *)
    echo "Voce nao sabe o que sao letras e digitos" ;;
esac
```

Gerando sequências para for (1)

- O comando `for` itera sobre uma lista de itens, que não precisa ser uma sequência numérica

```
for f in *.c *.h ; do
    ...
done
```

- Existem diferentes formas de gerar sequências numéricas

	1 a 10	10 a 1	2 a 10, passo 2
específica do Bash	{1..10}	{10..1}	{2..10..2}
seq	seq 10	seq 10 -1 1	seq 2 2 10
jot	jot 10	jot 10 10 1	jot 5 2 10

- ▶ `seq -s " "` para mudar o separador (default é `\n`)
- ▶ `jot`: primeiro argumento é a quantidade de números gerados

Gerando sequências para for (2)

- O Bash faz a expansão de chaves ({1..10}) antes de outras expansões e substituições
 - não é possível parametrizar o laço com uma variável ({1..\$n})
 - se a expansão gerar muitos elementos, usar chaves pode ser lento e consumir muita memória
- Outra opção específica do Bash é usar o chamado laço aritmético:

```
for ((i = 1; i <= n; i++)) ; do  
    echo $i  
done
```

 - considerada melhor do que expansão de chaves

Processando linhas com read

```
i=1  
# o 'IFS=' no read preserva os espaços da entrada  
cat "$1" | while IFS= read -r linha  
do  
    printf "%5d %s\n" $i "$linha"  
    ((i++))  
done
```